Algorithm 3: Quick Sort with Majority Voting

Input: Dataset D, sorter LLMSort$(\cdot)$, votes $v \geq 1$
Output: Sorted Dataset S
if $|D| \leq 1$ **then**
  return D
Choose pivot $p \leftarrow D[1]$;
Initialize $L \leftarrow [\,]$, $G \leftarrow [\,]$ `// final partitions`
Initialize $L_{initial} \leftarrow [\,]$, $G_{initial} \leftarrow [\,]$
$\mathcal{R} \leftarrow \{(x, \text{LLMSort}(p, x)) \text{ for } x \in D[2:]\}$;
for *each* $(x, r) \in \mathcal{R}$ **do**
  if $r = l$ **then** $L_{initial} \leftarrow L_{initial} \cup \{x\}$;
  else $G_{initial} \leftarrow G_{initial} \cup \{x\}$;
for *each* $(x, r_{initial}) \in \mathcal{R}$ **do**
  `// Select peer pool`
  if $r_{initial} = l$ **then** $P \leftarrow G_{\text{initial}}$;
  else $P \leftarrow L_{\text{initial}}$;
  $\mathcal{Y} \leftarrow \text{Sample}(P, v-1)$;
  $peer_results \leftarrow \{r_{\text{initial}}\} \cup \{\text{LLMSort}(x, y) \mid y \in \mathcal{Y}\}$;
  if `all`*(peer_results = l)* **then**
    $L \leftarrow L \cup \{x\}$;
  if `all`*(peer_results = g)* **then**
    $G \leftarrow G \cup \{x\}$;
  Record $(x, r_{\text{initial}}, \mathcal{Y}, peer_results)$ in $\mathcal{R}'$;
while $|L| + |G| < |D| - 1$ **do**
  $deadLock \leftarrow$ True;
  for *each* $(x, r_{initial}, peers, peer_results)$ *in* $\mathcal{R}'$ **do**
    if $x \in L \vee x \in G$ **then**
      continue
    if `all`*(peers* $\in L \cup G$*)* **then**
      $deadLock \leftarrow$ False;
      Alg 2$(x, r_{\text{initial}}, peers, peer_results) \cup \{x\}$;
  if *deadLock* **then**
    Alg 2$(x, r_{\text{initial}}, peers, peer_results) \cup \{x\}$;
$S_L \leftarrow \text{LLMQuickSort}(L, \text{LLMSort}, v)$;
$S_G \leftarrow \text{LLMQuickSort}(G, \text{LLMSort}, v)$;
return $S_L \parallel [p] \parallel S_G$;

overlapping groups of keys (i.e., $\frac{m}{2}$ keys), so that $\frac{m}{2}$ items of higher rank gradually 'bubble up' in successive windows.

Prompt Block 2: Comparison Prompt for Passage Ranking

```
You are given a question and two passages. Determine which
     passage answers the question better.
Question: {question}
Passage A: {key1}
Passage B: {key2}
```

In this subsection, we introduce two new comparison-based algorithms: (i) **quick sort with majority voting** to improve sorting quality compared to vanilla quick sort, and (ii) an **LLM-based external merge sort** inspired by the classical two-way external merge sort to reduce cost compared to external bubble sort. Later in the evaluations, we empirically demonstrate the effectiveness of these methods, showing that our quick sort variant leads to better accuracy than vanilla quick sort and the external merge sort yields substantial cost savings.

Algorithm 4: LLM External Merge Sort

Input: Dataset D, batch size m, sorter LLMSort$(\cdot)$,
  two-way merger LLMMerge$(\cdot)$
Phase 1: Run generation;
Partition D into chunks $C_1, \ldots, C_m$ of size at most m;
for $i \leftarrow 1$ **to** m **do in parallel**
  $S_i \leftarrow \text{LLMSort}(C_i)$ `// listwise sorting`
$Runs \leftarrow [S_1, \ldots, S_m]$;
Phase 2: Iterative merging;
while $|Runs| > 1$ **do**
  $NewRuns \leftarrow [\,]$;
  for $i = 1, 3, 5, \ldots, |Runs|$ **do**
    if $i + 1 \leq |Runs|$ **then**
      $S \leftarrow \text{LLMMerge}(Runs[i], Runs[i{+}1], m)$
        `// Algorithm 5`
      append S to $NewRuns$;
    else
      append $Runs[i]$ to $NewRuns$ `// odd run carried forward`
  $Runs \leftarrow NewRuns$;
return $Runs[1]$ `// final sorted sequence`

As shown in Algorithm 3, LLM quick sort algorithm follows the classical divide-and-conquer strategy, and uses an LLM comparator (LLMSort) which takes in two keys and then determine the semantic order between these two keys. Prompt 2 shows the template used to generate the correct order between two passages. Unlike vanilla quick sort, which relies on a single comparison against the pivot to partition items, our approach improves robustness by validating the partition through majority voting. In this method, an item is compared not only with the pivot but also with a set of sampled 'peers'. The core intuition relies on transitivity: if a key x is less than the pivot p, it should logically be less than any item y that is greater than p. Items that achieve full consensus across all peer comparisons are partitioned immediately. Items with conflicting results are resolved iteratively; they wait until their peers are firmly classified into the Less (L) or Greater (G) sets, then a voting routine (Algorithm 2) determines their final placement. This ensures that high-confidence decisions anchor the partition structure before uncertain cases are resolved. Only when a deadlock occurs (i.e., no remaining unclassified items have fully classified peers), we trigger the voting routine immediately and utlize the current partial partitions to resolve the cyclic dependencies.

To address the limitation of high running cost in external bubble sort proposed in RankGPT [58], we propose a new semantic external merge sort approach inspired by the classical external merge sort approach [27]. As shown in Algorithm 4, our approach operates in two distinct phases. First, the Run Generation phase divides the input into chunks of size m, which are individually

Algorithm 5: LLM Two-Way Merge

Input: Sorted runs L_1, L_2; batch size m; sorter LLMSort$(\cdot)$
$i \leftarrow 0,\ j \leftarrow 0$ `// indices in` L_1 `and` L_2
$S \leftarrow [\,]$;
$h \leftarrow \max(\lfloor m/2 \rfloor, 1)$;
while $i < |L_1|$ ***or*** $j < |L_2|$ **do**
  if $i \geq |L_1| \ \vee\ j \geq |L_2|$ **then**
    append $(i \geq |L_1|\ ?\ L_2[j{:}]\ :\ L_1[i{:}])$ to M; **break**
  `// Store` h `items from` L_1 `and` L_2
  $B \leftarrow$ up to h items from $L_1[i:]$ and $L_2[j:]$;
  $R \leftarrow$ LLMSort(B)
  `// Emit in order until one side is exhausted`
  $e_1 \leftarrow 0,\ e_2 \leftarrow 0$;
  for *each* $x \in R$ **do**
    append x to S;
    if $x \in L_1$ **then** $e_1 \leftarrow e_1 + 1$;
    else $e_2 \leftarrow e_2 + 1$;
    if $e_1 = h$ ***or*** $e_2 = h$ **then**
      break
  $i \leftarrow i + e_1, \quad j \leftarrow j + e_2$;
return M

sorted using LLMSort with Prompt 3 to derive the LLM-sorted permutation. Second, the Merge Phase (Algorithm 5) iteratively merge two sorted runs into one sorted run. The two-way merge operation employs a sliding-window mechanism: it buffers m keys ($m/2$ keys from each sorted run) for the LLMSort to derive a partial ordering on this buffer. From this local ranking, we extract items and add them to the final output until the candidates from one of the input runs are exhausted within the current window. We must re-fill this buffer again before the next extraction. This is because we need to compare the unseen element from the exhausted run with the remaining items in the buffer to determine the total order. This process continues until one of the input runs is completely exhausted and the other input run is append to the final output; hence, we obtain a merged sorted run.

Prompt Block 3: External Comparison Prompt for Passage Ranking

```
You are given a question and a list of passages.
Rank the passages based on how well they answer the question
     , from worst to best.
Question: {question}
Passages:
[1] {passage_1}
[2] {passage_2}
...
```

3.3 Limit K

A common usage pattern is to combine the **ORDER BY** clause with **LIMIT K**, where the user wants to retrieve the first K items from the sorted results. This pattern offers an additional opportunity to save the number of LLM invocations and costs. For example, the external bubble sorting algorithm [58], effectively finding the first K sorted data points, reduces the number of calls to LLM from $O(N^2/m^2)$ to $O(KN/m^2)$. See Table 1 for the number of LLM function calls used in each algorithm in both full sort and LIMIT K. Let's assume the user wants to retrieve the first K rows in descending order based on some criteria (e.g, retrieve the K most relevant passages to the user question). In quick sort, after it separate rows into L (lesser) and G (greater) lists comparing each row to the pivot and $v-1$ validation samples, if G contains more than K rows, then only G need to be sorted and L can be discarded. This is also known as partial quick sort [38]. It can be seen as a quick select, which uses $O(N)$ comparisons to find the partition with K rows, and then apply quick sort on K rows. For External Merge Sort (Algorithm 4 and Algorithm 5), it still performs the run generation phase (i.e., generating a sorted run with m rows) and then perform 2-way merge. During the 2-way merge, if the merged list reaches K items, then it can discard all the rest. As a result, having the LIMIT K clause also reduces the number of invocations for external merge sort. For the first $O(log\frac{K}{m})$ rounds, each run size will be less than K, and each round requires $O(\frac{N}{m})$ function calls. Once the run size reaches K, the behavior changes. For all subsequent rounds, we merge two runs of size K but strictly cap the output at K items (discarding the remaining K). As a result, the total number of items decrease by a factor of 2 for each round. Hence, the LLM function calls decrease from $\frac{N}{m}$, to $\frac{N}{2m}$, to $\frac{N}{4m}$, etc. This is a gemoetric sequence which is upper bound by $O(\frac{N}{m})$. The total LLM function calls can be approximated as $O(\frac{N}{m}(2 + log\frac{K}{m}))$.

	LLM Calls	
Algorithm	Full Sort	Limit K
Pointwise	$O(N)$	$O(N)$
Ext Pointwise	$O(N/m)$	$O(N/m)$
Quick Sort (v)	$O(vNlogN)$	$O(v(N + KlogK))$
Ext Bubble	$O(N^2/m^2)$	$O(KN/m^2)$
Ext Merge	$O(\frac{N}{m}(1 + log\frac{N}{m}))$	$O(\frac{N}{m}(2 + log\frac{K}{m}))$

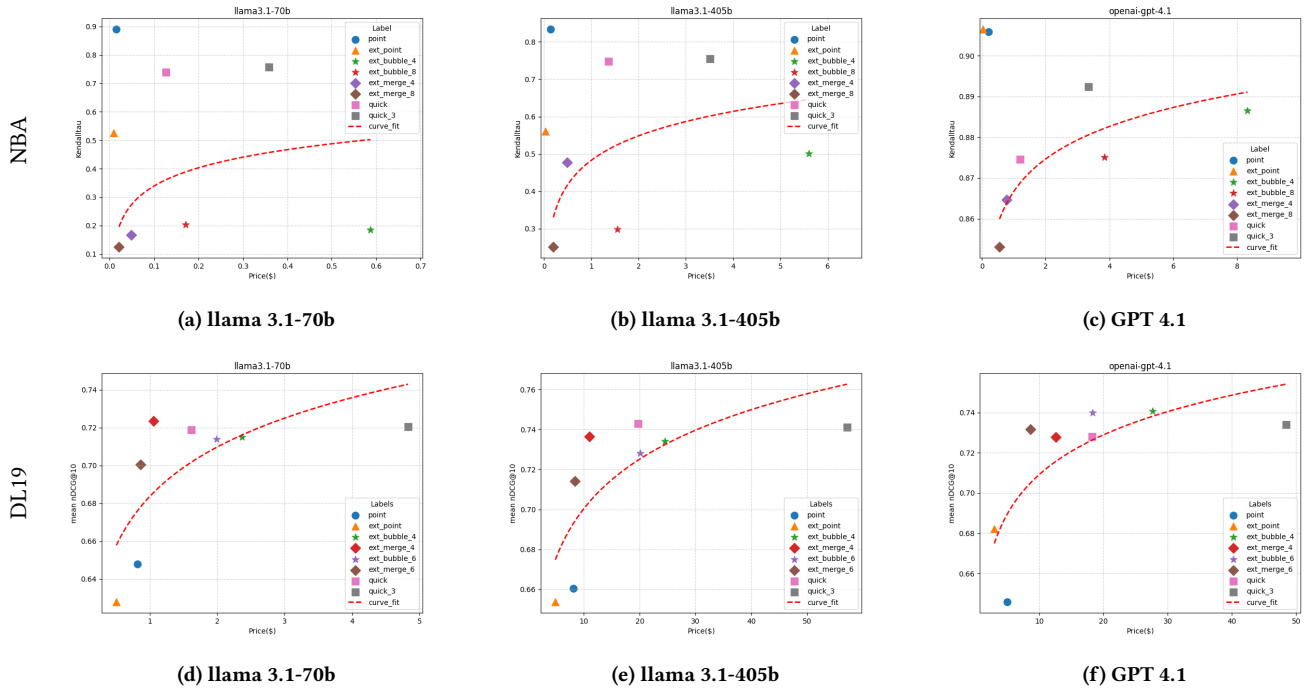
Table 1: LLM call complexity by algorithm, where N is the number of keys, m is batch size, and v is votes.

4 INITIAL EVALUATION

In this section, we evaluate our proposed algorithms alongside existing baselines on two different datasets. We report results in terms of both effectiveness (ranking quality) and cost (price), highlighting the trade-offs between value-based and comparison-based approaches. The main question we would like to answer in this section is that: **Is there a single sorting algorithm that always gives the best ranking quality?**

Datasets. The two dataset we used are:

- NBA: Contains 200 player names [43] to be ordered by height. In the semantic order by task, the height is masked, requiring the sort operation to be performed based solely on the player names.



(a) llama 3.1-70b (b) llama 3.1-405b (c) GPT 4.1

(d) llama 3.1-70b (e) llama 3.1-405b (f) GPT 4.1

Figure 1: Sorting accuracy vs. the monetary budget ($). While more budget generally leads to better accuracy, these figures often display a log-linear growth pattern (red curve), with accuracy increasing quickly at first before plateauing.

- DL19: TREC-DL19 is a widely used benchmark dataset in information retrieval literature [11]. The test sets contains 43 queries and 8.8 million passages. The goal is to rank the passages according to their relevance to each query.

4.1 Implementation and Metrics

In our experiments, all LLM models (Llama3.1-70b, Llama3.1-405b, and OpenAI GPT 4.1) are directly accessed through Snowflake Cortex REST API. For all experiments, we use the structured API and set the temperature to 0, which adopted greedy decoding to obtain the most deterministic outputs [41]. The input JSON schema was designed following the best practices of chain-of-thought prompting [65], as recommended in the OpenAI structured outputs guide [42]. All algorithms were implemented in Python. The pointwise [31] (*point*) and external pointwise (*ext_point*) methods uses the rating scale prompt from [77]; quick sort (*quick*) and our proposed quick sort with majority voting (*quick_3*) follow the prompt design in [37, 48]; and external bubble sort and our proposed external merge sort utilize the prompt from [58].

In evaluating sorted-ness on the NBA dataset, we adopt the well-established kendall's tau metric [25]. It quantifies the correspondence between two rankings, with values near 1 indicating strong agreement and values near -1 indicating strong disagreement. In evaluating the sorted-ness on DL19, we adopt the standard *nDCG@10* metric [64]. In addition, we follow the convention of re-ranking the top 100 passages retrieved by BM25 for each query [49]. We directly use the `pyserini` toolkit [32] for BM25 retrieval. Normalized discounted cumulative gain (nDCG) yields values from 0.0 to 1.0, where higher values indicate rankings that are closer to the ideal order.

4.2 Observations

NBA Player Height Ranking. The first row in Figure 1 reports Kendall's tau score comparing predicted orderings with the ground truth. Overall, the *value-based* approach emerges as the top-performing algorithm across three models. The pointwise method consistently achieves high accuracy on all models, whereas external pointwise has higher variance. With GPT-4.1, the pointwise and the external pointwise achieve 0.91 accuracy, while external pointwise is cheaper. We find that our adaptive batching strategy is more effective on GPT-4.1 than Llama 3.1; the latter exhibited higher error rate in adhering to JSON constraints (e.g., returning incorrect item counts) or JSON decode error. These observations align with the findings reported in [68] in which OpenAI models are better than Llama models at producing structured formats. The high accuracy achieved by pointwise method also suggests that these LLMs, when applied to factual data (highly likely part of its training corpus), benefit from directly inferring values since they have perhaps memorized the statistics. The comparison based approaches consistently under-perform. Even though we have added a chain-of-thought reasoning step in the input JSON schema, it fails to bridge the performance gap, suggesting that the model's direct retrieval is simply more reliable than its explicit reasoning on these familiar data. While our proposed quick sort variant consistently outperforms the standard quick sort, other methods exhibit performance instability contingent on the model. For instance, external bubble